

## 1. The Weak / Vulnerable Structure

In a weak implementation, the URL typically contains static identifiers, unhashed tokens that never expire, or predictable sequences.

```
https://app.example.com/login/magic?user_id=4102&token=welcome123
```

### Why this is a weak implementation (Your screenshot annotations):

- **Predictable `user_id`:** Attackers can easily guess or brute-force adjacent user IDs (e.g., 4103, 4104) to attempt account takeover.
- **Static/Weak Token:** Using a simple, non-cryptographic string (`welcome123`) makes it susceptible to guessing or dictionary attacks.
- **Missing Safeguards:** There are no parameters dictating when the link expires or ensuring it can only be clicked once.

## 2. The Secure / Validated Structure

A secure production-ready magic link uses highly complex, cryptographically secure random tokens paired with server-side restrictions.

```
https://app.example.com/auth/callback?token=7f9a2c8e3b1d4f6a0c5e9b8d7a6f5e4d3c2b1a0f9e8d7c6b5a4f3e2d1c0b9a8&expires=1782049600&sig=a8f9e2...
```

### Why this is a secure implementation (Your screenshot annotations):

- **High-Entropy Token:** The token is a long, cryptographically random string (typically SHA-256 or a secure UUID) generated by a secure random number generator, making it impossible to guess.
- **Expiration Timestamp (`expires`):** A Unix timestamp embedded or tracked on the backend that strictly limits the link's validity window (usually 5 to 15 minutes).
- **Cryptographic Signature (`sig`):** An optional but excellent practice where the entire URL is signed using a secret server key. If an attacker attempts to alter the expiration time or the token, the signature becomes invalid, and the server rejects the request immediately.
- **One-Time Use (Backend Logic):** While not visible in the URL string itself, your portfolio note should mention that the backend database immediately deletes or marks this token as "consumed" the exact second it is resolved.